

ASSIGNMENT – 8

AIM: To convert an infix expression into postfix, construct its DAG, identify common subexpressions, and generate optimized three-address code.

Chosen expression:

$$((a + b) * (c - d)) + ((a + b) * (c - d))/e$$

ALGORITHM:

1. Read the infix expression.
2. Convert infix to postfix using stack:
 - Operands go directly to output.
 - Operators are pushed based on precedence.
 - Parentheses control grouping.
3. Construct DAG from postfix:
 - Create leaf nodes for operands.
 - Create operator nodes for operations.
 - Reuse an existing node if the same subexpression already exists.
4. Identify repeated subexpressions.
5. Generate optimized three-address code from DAG.
6. Compare unoptimized and optimized operations.

THEORY:

Infix to Postfix

Infix

$$((a + b) * (c - d)) + ((a + b) * (c - d))/e$$

Postfix

ab+cd-*ab+cd-*e/+

DAG Construction

Leaf Nodes

a, b, c, d, e

Operator Nodes

$N1 = a + b$

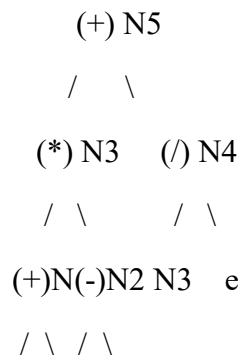
$N2 = c - d$

$N3 = N1 * N2$

$N4 = N3 / e$

$N5 = N3 + N4$

DAG Representation



a b c d

Common Subexpressions

The repeated subexpressions are:

$(a + b)$

$(c - d)$

$(a + b) * (c - d)$

Without DAG, these expressions are calculated twice.

With DAG, each common subexpression is calculated only once and reused.

Unoptimized Three-Address Code

$t1 = a + b$

$t2 = c - d$

$t3 = t1 * t2$

$t4 = a + b$

$t5 = c - d$

$t6 = t4 * t5$

$t7 = t6 / e$

$t8 = t3 + t7$

Total operations = 8

Optimized Three-Address Code Using DAG

t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / e
t5 = t3 + t4

Total operations = 5

CODE:

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

#define MAX 100

char stack[MAX];

int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
    return stack[top--];
}

int precedence(char op) {
    if (op == '*' || op == '/')
        return 2;
    if (op == '+' || op == '-')
        return 1;
    return 0;
}

void infixToPostfix(char infix[], char postfix[]) {
```

```

int i, j = 0;

char ch;

for (i = 0; infix[i] != '\0'; i++) {
    ch = infix[i];
    if (isalnum(ch)) {
        postfix[j++] = ch;
    } else if (ch == '(') {
        push(ch);
    } else if (ch == ')') {
        while (top != -1 && stack[top] != '(') {
            postfix[j++] = pop();
        }
        pop();
    } else {
        while (top != -1 && precedence(stack[top]) >= precedence(ch)) {
            postfix[j++] = pop();
        }
        push(ch);
    }
}

while (top != -1) {
    postfix[j++] = pop();
}

postfix[j] = '\0';
}

int main() {
    char infix[MAX] = "((a+b)*(c-d))+((a+b)*(c-d))/e";
    char postfix[MAX];

```

```

infixToPostfix(infix, postfix);

printf("Infix Expression: %s\n", infix);
printf("Postfix Expression: %s\n", postfix);
printf("\nOptimized Three Address Code:\n");

printf("t1 = a + b\n");
printf("t2 = c - d\n");
printf("t3 = t1 * t2\n");
printf("t4 = t3 / e\n");
printf("t5 = t3 + t4\n");

return 0;
}

```

OUTPUT:

```

Infix Expression: ((a+b)*(c-d))+((a+b)*(c-d))/e
Postfix Expression: ab+cd-*ab+cd-*e/+

Optimized Three Address Code:
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / e
t5 = t3 + t4

```

OBSERVATION: The expression contains repeated subexpressions. DAG detects these repeated parts and stores them only once. Therefore, redundant calculations are avoided.

Type	Number of Operations
Unoptimized Code	8
Optimized Code	5

Operations reduced = 3